Spire Labs DA Builder Contracts

July 2025

Made with ❤ by the following Creed authors: Valentin Quelquejay

Table of Contents

Executive Summary 4

Scope and Objectives 5

Audit Artifacts 6

Findings 8

- [Medium] `gasLimit` Not Verified by `TrustlessProposer` Allows Builder Gas-Tampering
- [Medium] In `TrustlessProposer`, Nonce Should Be Incremented Before External Call
- [Minor] Unused Variable in `ProposerMulticall.multicall()`
- [Minor] Missing Events for State-Changing Functions
- [None] Confusing Parameter Naming in `_charge` Function
- [None] Public Initializer Functions
- [None] Add a Security Disclaimer to Proposer Contract

Disclaimer 16

Executive Summary

This report presents the results of our engagement with Spire Labs to review their DA Builder smart contracts.

The review was conducted over 6 days, from July 28, 2025 to August 4, 2025 by Valentin Quelquejay. A total of 6 person-days were spent.

The review identified three findings, with two classified as medium severity and two classified as low severity. No critical or major findings have been identified. This report also includes three none-severity findings which are suggestions around code readability and maintainability, and information that should be kept in mind for safely operating the system.

The system is designed to operate with delegated user wallets via EIP-7702. Users should take all appropriate security measures to ensure secure interaction, including only delegating with addresses with restricted permissions.

Scope and Objectives

Our review focused on the commit hash `ad9cdb2417cad73b14bd067738a88661354effae`.

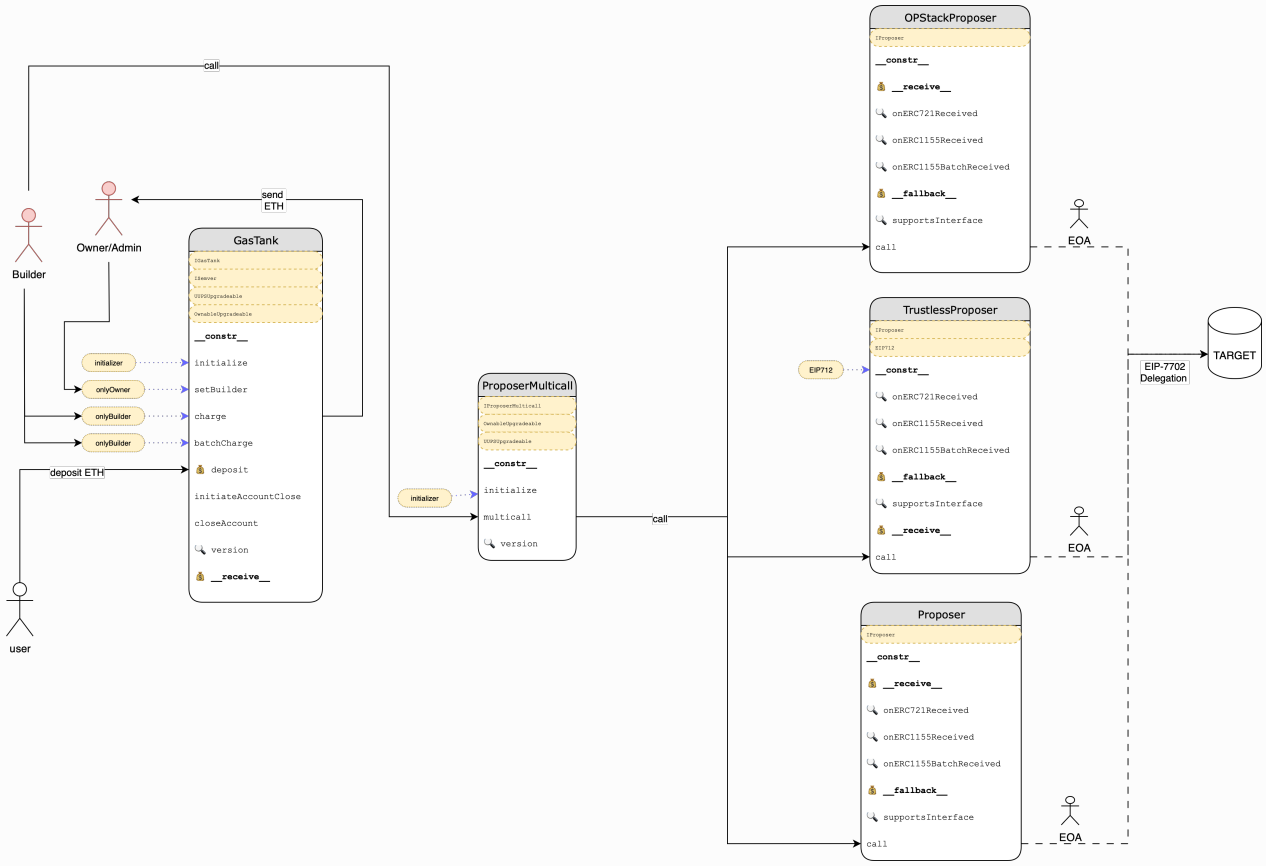
The client submitted two pull requests (PR#2 and PR#3) during the engagement, addressing issues we identified throughout the review. We reviewed these PRs as part of the engagement.

Together with the Spire Labs team, we identified the following priorities for our review:

- Funds cannot be withdrawn from the GasTank before the withdrawal delay is elapsed.
- Funds deposited in the GasTank can only flow to the Proxy Admin, or be refunded.
- Multicall function works in the expected way.
- Ensure that the system is implemented consistently with the intended functionality, and without unintended edge cases.
- Identify known vulnerabilities particular to smart contract systems, as outlined in our Smart Contract Security Field Guide, and the ones outlined in the EEA EthTrust Security Levels Specification.

Audit Artifacts

System Architecture



System Roles

- **Builder (EOA Hot Wallet)**: The builder aggregates and submits transactions and manages gas charging. The Builder's private key must be safeguarded with extreme care. Compromise of this key grants control over user funds and transaction execution.
- **ProxyAdmin (Multisig)**: The multisig controls administrative functions and receives charged fees. The multisig signers' private keys must be protected. Compromise allows unauthorized upgrades and fund access.
- **Users (EOA Accounts)**: End users of the aggregation service who can deposit/withdraw funds and delegate to proposer contracts via EIP-7702. Security level depends on proposer contract choice.

The security of this system heavily depends on the protection of private keys for the Builder and ProxyAdmin multisig. Any compromise of these keys might result in system compromise and loss of

user funds. Implement proper key management practices including multi-signature wallets and secure hot-wallet key storage solutions e.g., HSMs.

General Security Concerns and Precautions

Off-Chain Billing: All billing logic for gas and fees is off-chain, meaning the `GasTank` contract's charging mechanism is decoupled from actual execution. Users must trust the off-chain systems to behave correctly and charge accurately. Any compromise or malfunction in the off-chain billing system can result in incorrect charges.

EIP-7702 Delegation: The system is designed for EIP-7702 account delegation, where users delegate their EOA to proposer contracts. The `Proposer` contracts grants privileges to `ProposerMulticall`. We discourage the use of a trusted proposer with arbitrary call. If the Builder key is compromised, the attacker would gain complete control over user accounts. Users must be extremely careful and only delegate addresses that have no or very limited permissions and funds. Users must understand that proposer contracts can take actions on their behalf. Delegated addresses should never have admin rights in targeted systems. Users should favor trustless proposers and restrict proposers to a single functionality.

Findings

Medium `gasLimit` Not Verified by `TrustlessProposer` Allows Builder Gas-Tampering

Fixed

> This issue was addressed in PR#3.

The `gasLimit` field in the `Call` struct consumed by `ProposerMulticall.multicall()` is set unilaterally by the `BUILDER` and is not included in the user-signed EIP-712 digest validated in the `TrustlessProposer`. A malicious or malfunctioning builder can therefore set an arbitrary `gasLimit` diverging from user's intent and undetectable by off-chain verifiers. For instance, one could raise `gasLimit` above the user's expectation, letting a target call consume unbounded gas at the sponsor's expense.

Example

contracts/src/contracts/ProposerMulticall.sol

```
function multicall(
  Call[] calldata _calls
) external {
```

contracts/src/interfaces/IProposerMulticall.sol

```
  struct Call {
    address proposer;
    address target;
    bytes data;
    uint256 value;
    uint256 gasLimit;
  }
```

contracts/src/contracts/proposer/TrustlessProposer.sol

```
(bytes memory _sig, uint256 _deadline, uint256 _nonce, bytes memory _calldata) =
  abi.decode(_data, (bytes, uint256, uint256, bytes));
```

Recommendation

Add `gasLimit` to the data covered by the EIP-712 signature. This will ensure the `gasLimit` provided to `multicall` matches the users's intent, or the function will revert. This cryptographically binds execution cost to what the user explicitly approved and removes the builder's ability to tamper with gas.

Medium In `TrustlessProposer`, Nonce Should Be Incremented Before External Call

Fixed

> This issue was fixed by the team in PR#2.

In `TrustlessProposer.call()`, the `nestedNonce` is incremented after the external call to `_target`, which violates the checks-effects-interactions pattern. While the re-entrancy risk is lower in this context given the access control in place, there is still a risk of signature replay via same-transaction re-entrancy if the `ProposerMulticall` / `Builder` ever went rogue. Thus, we strongly recommend incrementing the nonce before any external call to prevent potential issues and follow security best practices.

How an attack could unfold:

1. Malicious `ProposerMulticall` (PM) under Builder control The Builder (an ERC-7702 account) invokes `PM.multicall()`.
2. First call into `TrustlessProposer` (TP) PM forwards a signed message with nonce N: `PM → TP.call(target = MaliciousTarget, nonce = N)`
3. External interaction
   TP forwards execution to `MaliciousTarget` (MT), which steals tokens and then calls the Builder.
4. In-transaction re-entrancy The Builder, still executing contract code in the same transaction, re-enters PM, which again calls
   `TP.call(target = MT, nonce = N)` before TP has incremented `nestedNonce`.
5. Nonce reuse, theft repeats Because `nestedNonce` is still N, the second call passes validation and MT can siphon assets again. The loop can repeat until gas runs out.

The attack is only possible because the nonce update happens after the external call:

```
(bool _success,) = _target.call{value: _value}(_calldata);  // interaction first
...
unchecked { nestedNonce = _currentNonce + 1; }              // effect later
```

We could even assume that a legitimate contact is upgraded to a MaliciousTarget (MT) in the same multi-call transaction. This wouldn't completely mitigate against such an attack, but could reduce the impact.

Recommendation

Move the nonce increment ahead of the external interaction (i.e., follow checks-effects-interactions) to guarantee a signature cannot be replayed within the same transaction.

Minor Unused Variable in `ProposerMulticall.multicall()`

Fixed

> Fixed in PR#4.

The `multicall` function in the contract `ProposerMulticall` declares a variable `_msgValueSummation` that accumulates the sum of all call values but is never used after.

Examples

contracts/src/contracts/ProposerMulticall.sol

```
function multicall(
  Call[] calldata _calls
) external {
  if (msg.sender != BUILDER) revert Unauthorized();
  uint256 _callsLength = _calls.length;
  uint256[] memory _internalCallsGasUsed = new uint256[](_callsLength);

  uint256 _msgValueSummation;
  uint256 _preCallGasLeft;
  uint256 _gasUsed;
  bytes memory _callData;

  for (uint256 i; i < _callsLength; ++i) {
    _msgValueSummation += _calls[i].value;
```

Recommendation

Remove the unused variable to clean up the code and reduce gas usage. Alternatively, if value validation or monitoring was intended, implement proper validation logic using the calculated sum or emit the value for transparency

Minor Missing Events for State-Changing Functions

Partially Fixed

> The team partially addressed this issue in PR#4. They chose to emit an event in `setBuilder`, but left the `_charge` function unchanged because it is called frequently and they prefer to save gas.

Several state-changing functions in `GasTank` do not emit events. It is strongly recommended that all state-changing functions emit events. It helps off-chain systems and users to monitor the system's state, more particularly accounting in this specific case.

Examples

contracts/src/contracts/GasTank.sol

```
function setBuilder(
  address _builder
) external onlyOwner {
  builder = _builder;
}
```

contracts/src/contracts/GasTank.sol

```
function _charge(Account calldata _account, address _proxyAdmin) internal {
  balances[_account.operator] -= _account.charge;
  (bool _success,) = _proxyAdmin.call{value: _account.charge}('');
  if (!_success) {
    revert FailedLowLevelCall();
  }
}
```

Recommendation

Consider emitting events on state-changing functions. This will improve transparency, enable better off-chain monitoring, and provide a complete audit trail for all state changes.

None Confusing Parameter Naming in _charge Function

The `_charge` function in `GasTank` has a misleading parameter name `_proxyAdmin` which represents the recipient of the charged funds. While the proxy admin does turn out to be the recipient of funds in this case, the parameter name should reflect its role in the function rather than its broader system role for better clarity.

contracts/src/contracts/GasTank.sol

```
function _charge(Account calldata _account, address _proxyAdmin) internal {
  balances[_account.operator] -= _account.charge;
  (bool _success,) = _proxyAdmin.call{value: _account.charge}('');
```

Recommendation

Rename the parameter to better reflect its purpose, .e.g. `_recipient`. This makes the code more readable and self-documenting by clearly indicating that the address parameter receives the charged funds.

None Public Initializer Functions

Several contracts in the codebase have `public` initializer functions that can be called by anyone. This issue serves as a reminder that all deployment scripts should be atomic to prevent initialization front-running issues.

Examples

contracts/src/contracts/GasTank.sol

```
function initialize(address _owner, address _builder) external virtual override …
  __Ownable_init(_owner);
```

contracts/src/contracts/ProposerMulticall.sol

```
function initialize(
  address _owner
) external virtual initializer {
  __Ownable_init(_owner);
}
```

None Add a Security Disclaimer to Proposer Contract

The `Proposer` contract provides full arbitrary call privileges to the `ProposerMulticall` contract, which represents a significant security risk if not used carefully. This contract should include a disclaimer warning users about the security implications and proper usage guidelines. As soon as an EOA authorizes the proposer contract via EIP-7702, the function `call` allows the `ProposerMulticall` contract to execute any arbitrary call on behalf of the account.

Recommendation

Add a security disclaimer to the contract documentation and comments to make sure end-users implement appropriate security measures when integrating with this contract e.g.,

```
/// @title Proposer
///
/// @dev ⚠  SECURITY WARNING: This contract provides FULL ARBITRARY CALL PRIVILEGES
///      to the ProposerMulticall contract. Use with extreme caution.
///
/// @dev SECURITY REQUIREMENTS:
///      - Only use with EOA accounts that have NO privileges or VERY RESTRICTED privileges
///      - Do NOT use with accounts that have admin roles, ownership, or significant permissions
///
/// @dev An example implementation of a proposer contract that is compatible with ...
```

File Hashes

- ./contracts/script/Deposit.s.sol
  - 529a62be5a25bf0f11ff10abc84ef9f9b5403b67405cc59d747e3b054e2e0da0
- ./contracts/script/Deploy.s.sol
  - 268f102934565c2bafba598a524875b5c99b54463d1a42a07b27bd40428552c4
- ./contracts/src/contracts/proposer/TrustlessProposerEntry.sol
  - 2eb868f134bfd37f70352c096417b8cfde4612783792e75a134b156100884fd0
- ./contracts/src/contracts/proposer/OPStackProposer.sol
  - fcfa10be4bc6fbb75adec335abf4136af0830730b0b08e7f161ec238a140b6ce
- ./contracts/src/contracts/proposer/TrustlessProposer.sol
  - 597ee605e8cab003fb25bd0a52d4300551f2c142d2d5832ca347fcbbcf4baa8e
- ./contracts/src/contracts/proposer/Proposer.sol
  - fb4c1211912c37879a2adff980e45e87d5f5bfe0e9ed6f0e89e6aa48116c0c5c
- ./contracts/src/contracts/ProposerMulticall.sol
  - c4afcabdcfd75446e0c61aadb737e7233979a16264e5d5b6cf77ddb0b64b4a35
- ./contracts/src/contracts/GasTank.sol
  - e74e8b4e8fa6843545d480c3c82cd04dd3f929c7efea1339178552c0583b4c61

Disclaimer

Creed ("CD") typically receives compensation from one or more clients (the "Clients") for performing the analysis contained in these reports (the "Reports"). The Reports may be distributed through other means, including via CD publications and other distributions.

The Reports are not an endorsement or indictment of any particular project or team, and the Reports do not guarantee the security of any particular project. This Report does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. No Report provides any warranty or representation to any Third-Party in any respect, including regarding the bugfree nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the Reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset. Specifically, for the avoidance of doubt, this Report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. CD owes no duty to any Third-Party by virtue of publishing these Reports.

PURPOSE OF REPORTS The Reports and the analysis described therein are created solely for Clients and published with their consent. The scope of our review is limited to a review of code and only the code we note as being within the scope of our review within this report. Any Solidity code itself presents unique and unquantifiable risks as the Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond specified code that could present security risks. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. In some instances, we may perform penetration testing or infrastructure assessments depending on the scope of the particular engagement.

CD makes the Reports available to parties other than the Clients (i.e., "third parties") – on its website. CD hopes that by making these analyses publicly available, it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.

LINKS TO OTHER WEB SITES FROM THIS WEB SITE You may, through hypertext or other computer links, gain access to web sites operated by persons other than CD. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that CD are not responsible for the content or operation of such Web sites, and that CD shall have no liability to you or any other person or entity for the use of third party Web sites. Except as described below, a hyperlink from this web Site to another web site does not imply or mean that CD endorses the content on that Web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the Reports. CD assumes no responsibility for the use of third party software on

the Web Site and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.

TIMELINESS OF CONTENT The content contained in the Reports is current as of the date appearing on the Report and is subject to change without notice. Unless indicated otherwise, by CD.